NATIONAL AND KAPODISTRIAN UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Bantak Emil*
ID: sdi1800122

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*
Semester: *Spring Semester 2025-2026*

Contents

1 Abstract **3**

2 Data processing and analysis **3**
2.1 Data Cleaning . . . 3
2.2 Analysis . . . 3

3 Experiments **6**
3.1 Experiment 1 — Baseline Experiment . . . 6
3.1.1 Model Configuration . . . 6
3.1.2 Note on Implementation Provenance . . . 6
3.1.3 Pipeline Mechanics . . . 11
3.1.4 Results . . . 12
3.1.5 Results Analysis and Explanation . . . 12
3.2 Experiment 2 — Combination 1: Question Understanding . . . 13
3.2.1 Question Intent Agent — Changes . . . 14
3.2.2 Gap and Evasion Agent — Changes . . . 14
3.2.3 Generation Settings Change . . . 15
3.2.4 Results . . . 15
3.2.5 Results Analysis and Explanation . . . 16
3.3 Experiment 3 — Combination 2: Answer Analysis . . . 17
3.3.1 Answer Content Agent — Changes . . . 17
3.3.2 Gap and Evasion Agent — Changes . . . 18
3.3.3 Generation Settings . . . 18
3.3.4 Results . . . 18
3.3.5 Results Analysis and Explanation . . . 19
3.4 Experiment 4 (Best Experiment) — Combination 3: Pipeline Connection 20
3.4.1 Gap and Evasion Agent — Changes . . . 20
3.4.2 Decision Agent — Changes . . . 21
3.4.3 Generation Settings . . . 22
3.4.4 Results . . . 22
3.4.5 Results Analysis and Explanation . . . 22

4 Comparison with Previous Assignments **24**
4.1 Vector-Based Classifier (Assignment 1) 24
4.2 Encoder Only Transformer Model (Assignment 2) 25
4.3 Single LLM Prompting (Assignment 3) 26

5 Overall Comparison Across Assignments **27**
5.1 Combined Summary 27
5.2 Per-Class Comparison (Test Set) 28
5.3 Failure Part Comparison 28
5.4 Best Performing Approach 29

1. Abstract

This report presents the development and evaluation of a lightweight multi-agent prompting framework, D3-Agentic Prompting, for response clarity classification in political question–answer (QA) pairs. The task uses the dataset released for SemEval 2026 Task 6 (CLARITY) and requires predicting whether a president's answer to a journalist's question is a Clear Reply, an Ambivalent response, or a Clear Non-Reply.

The D3-Agentic Prompting decomposes the classification task into four specialized agents, a Question Intent Agent, an Answer Content Agent, a Gap and Evasion Agent and a Decision Agent, each performing a distinct subtask and passing structured intermediate output to the next stage of the pipeline. The framework is implemented using Qwen3.5-0.8B from the Hugging Face Transformers ecosystem, with greedy decoding and a fixed random seed to ensure reproducibility.

A baseline implementation of the four-agent architecture is first established, after which a series of experiments systematically extends individual agents with additional output fields and corresponding prompt instructions, in order to test whether richer intermediate information improves classification performance. The resulting D3-Agentic Prompting system is compared against the previous assignments, using weighted F1 score as the primary metric and macro F1 score reported as an additional metric. Each experiment is accompanied by a detailed analysis explaining the observed results in relation to the specific prompt and pipeline modifications made. Prior to experimentation the dataset undergoes data cleaning.

2. Data processing and analysis

2.1. Data Cleaning

The first cleaning step involves identifying and removing (question, answer) pairs with conflicting clarity labels. Since the dataset was annotated by multiple annotators the same (question, answer) pair could receive different clarity labels from different annotators. Keeping such pairs would introduce noise into the training process, so for the same (question, answer) pair if the annotators did not agree on the same clarity label this pair is being dropped. This cleaning step is applied only to the train set where 12 conflicting pairs were found as the test set contains no conflicting pairs.

The second cleaning step removes rows marked as inaudible (inaudible = True). This means that during the interview the audio quality was poor or unclear at that specific point so the annotator could not hear the answer properly making these entries useless for training the classifier. In the train set 45 such rows were found and dropped.

2.2. Analysis

The exploratory data analysis is performed on the train set and on the test set after the data cleaning steps. The train set contains 3,379 samples and the test set contains 308 samples covering interviews from four U.S. presidents: Donald J. Trump, Barack Obama, George W. Bush and Joseph R. Biden.

Label Distribution
The dataset is significantly imbalanced across all three classes. In the train set, which contains 3,379 samples the Ambivalent class dominates with 2,001 samples (59.2%), followed by Clear Reply with 1,029 samples (30.5%) and Clear Non-Reply with only 349 samples (10.3%). The same imbalance pattern is observed in the test set, which contains 308 samples where Ambivalent accounts for 206 samples (66.9%), Clear Reply for 79 samples (25.6%) and Clear Non-Reply for only 23 samples (7.47%). This class imbalance is an important consideration for model training as a classifier that simply predicts Ambivalent for every sample would already achieve a high accuracy. This is why weighted F1-score is used as the primary evaluation metric instead of accuracy, as it accounts for class imbalance by weighting each class proportionally to its support.

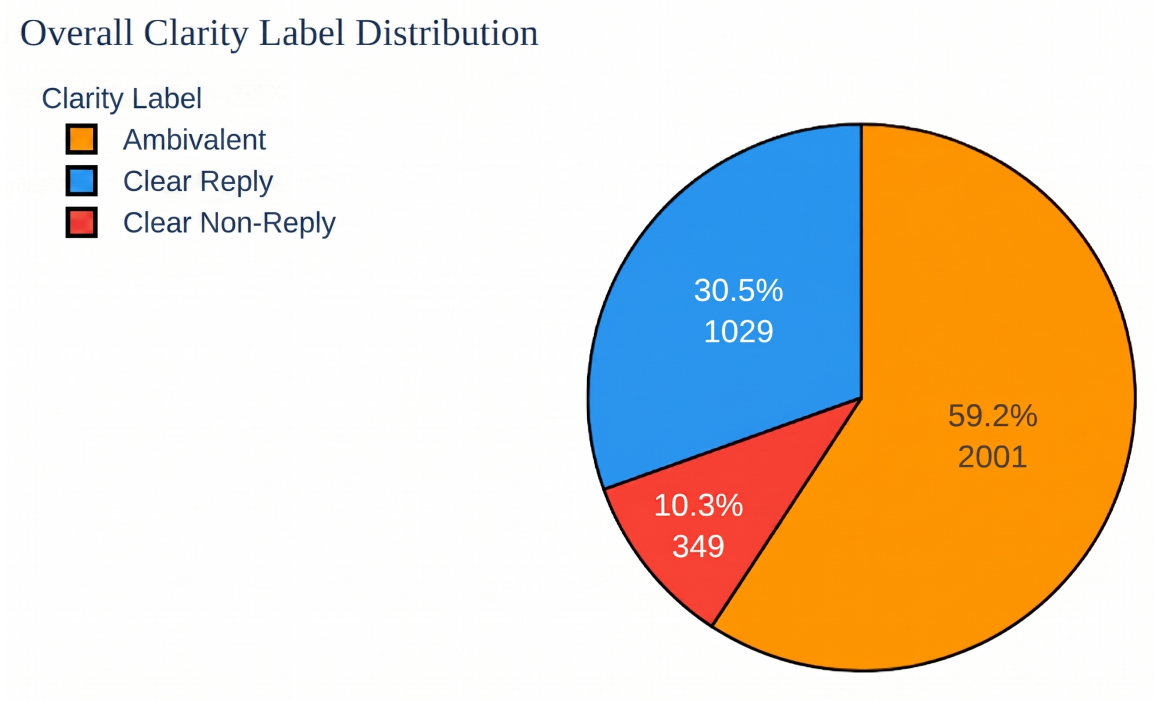


Figure 1: Label distribution in the training set

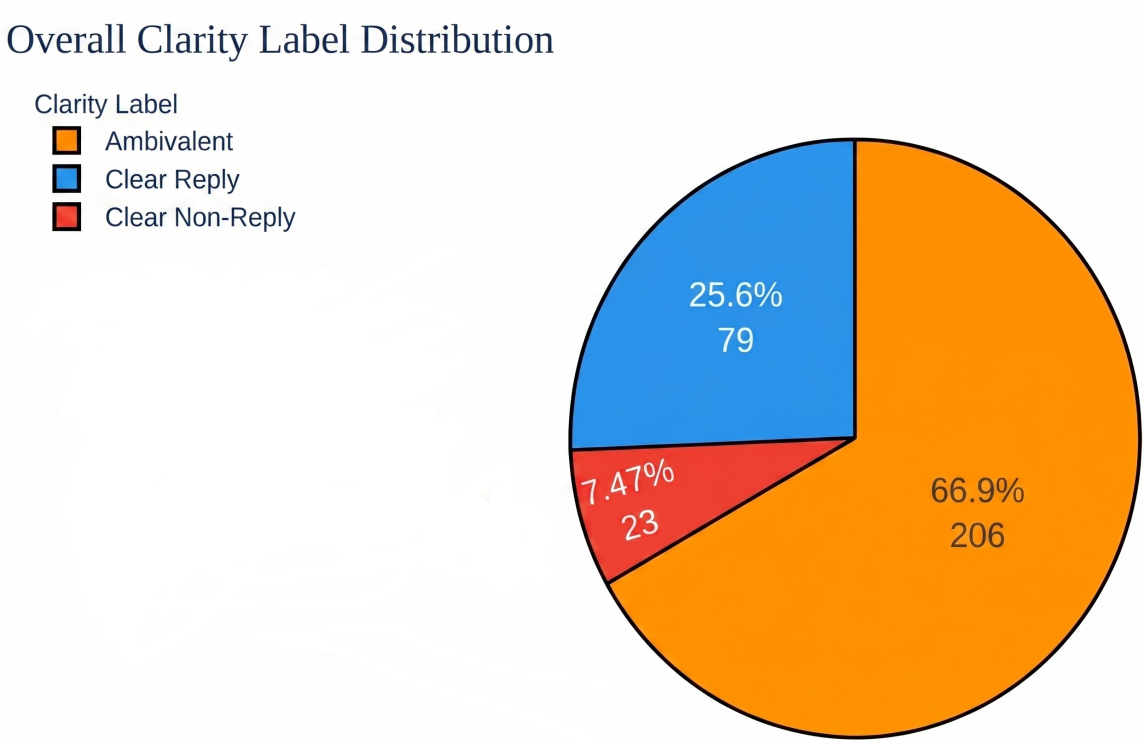


Figure 2: Label distribution in the test set

Answer Length Statistics

The answer length statistics reveal meaningful differences across the three clarity labels. Ambivalent answers are on average the longest with a mean of 327.9 words and a median of 241.0 words suggesting that when presidents give ambivalent responses they tend to speak at length without fully committing to a clear answer. Clear Reply answers have an average of 267.9 words and a median of 190.0 words. Clear Non-Reply answers are notably shorter with an average of only 136.0 words and a very low median of 40.0 words indicating that when presidents choose not to answer a question they tend to do so with very few words rather than giving a long evasive response.

Label	Average Length	Median	Min	Max
Clear Reply	267.9	190.0	1	1592
Ambivalent	327.9	241.0	1	2117
Clear Non-Reply	136.0	40.0	1	1407

Table 1: Answer length statistics per label on the training set

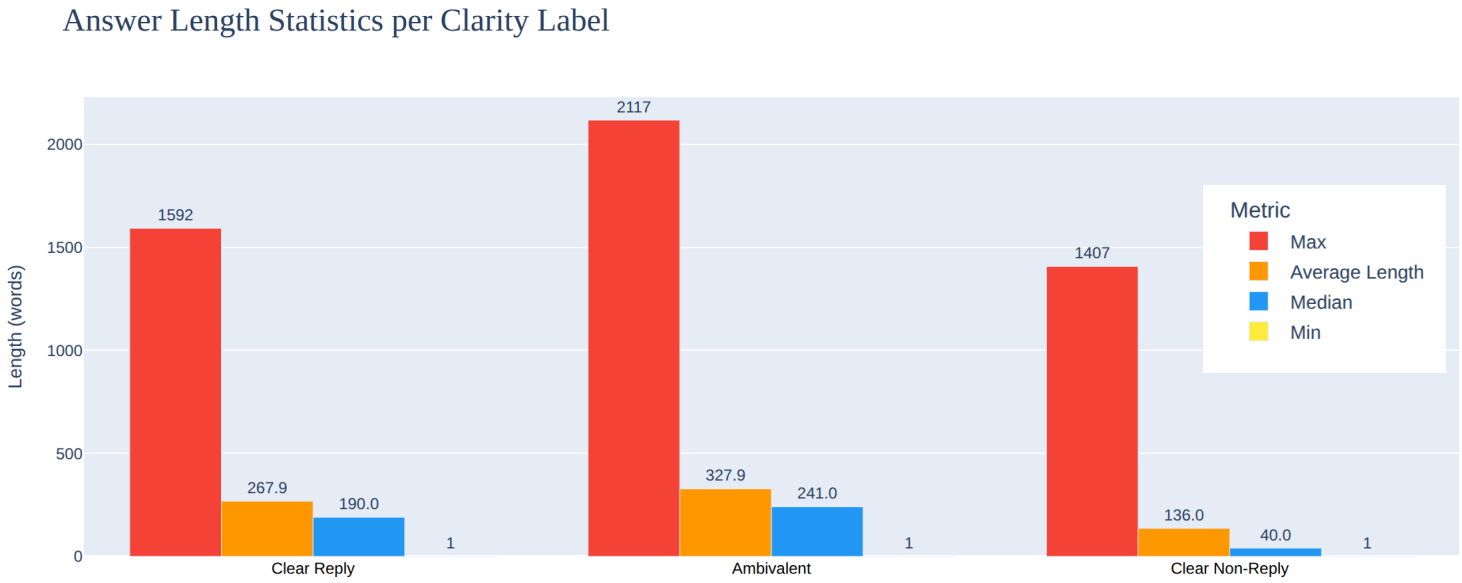


Figure 3: Answer length distribution by clarity label in the training set

3. Experiments

3.1. Experiment 1 — Baseline Experiment

3.1.1. Model Configuration.

Model: Qwen3.5-0.8B, greedy decoding (do_sample=False), seed = 42, max_new_tokens = 180 for the three intermediate agents and 90 for the Decision Agent.

3.1.2. Note on Implementation Provenance.

The agent prompts and pipeline scaffolding for this baseline were adapted directly from the provided lab code, due to time constraints. I consider this a reasonable starting point because after reviewing the lab's implementation, I found its structure to be well designed and robust, encoding several non trivial decisions, strict JSON only output schemas per agent, an explicit instruction to disambiguate the *target question* from the broader multi part interview question and a Decision Agent that combines rule based heuristics with five few-shot calibration examples. Given the strength of this scaffold, I decided to build directly on top of it rather than reimplementing it from scratch, allowing us to focus our own experimentation (Experiments 2+) on systematically varying individual agent prompts and measuring the effect. Below, I explain the structure of each agent in detail.

Question Intent Agent

What I have
This agent reads only the TARGET_QUESTION, explicitly distinguished from the full, possibly multipart interview question and determines what information would be required for a clear answer. It is guided to reason in terms of question type i.e. whether the question asks for yes/no, what happened, why/how something happened, a specific action/plan/reason/stance/evaluation or information about a named entity and is constrained to return exactly one JSON object with the fields described below and nothing else.

Output fields

- `question_type`: categorical label for the kind of question (yes_no, what, why, how, when, where, who, stance, explanation, action_plan, evaluation, other).
- `question_intent`: one sentence describing what the target question asks.
- `must_address`: a list of specific requirements an answer must satisfy to be considered clear.
- `clear_answer_criteria`: one sentence describing what a clear answer must contain.
- `minimal_sufficient_answer`: a short description of the minimum information needed for a Clear Reply.

Why I chose this
The CLARITY dataset frequently embeds the target subquestion inside a longer interview question, so without explicitly isolating *what is actually being asked*, downstream agents risk penalizing answers for not covering unrelated parts of the question this motivates the dataset rule restricting analysis to the TARGET_QUESTION alone. The question-type taxonomy (yes/no, what, why/how, action/plan/stance/evaluation, named entities) was chosen to cover the recurring question forms observed in political interview transcripts, giving the agent a small, finite set of categories rather than open-ended reasoning about question types. Finally, constraining the output to a single, strictly formatted JSON object (rather than free text) was meant to give the next agent in the pipeline a concrete, parseable checklist to compare against, instead of a paragraph it would have to re-interpret and to make downstream parsing more robust against malformed generations.

Answer Content Agent

What I have
This agent reads the answer together with the TARGET_QUESTION and extracts only the content of the answer relevant to that target question, without deciding the final clarity label. It is guided to look for direct answer phrases (yes, no, I did, I did not, we

will, we will not, because...), concrete facts, actions, reasons, explanations, examples and commitments, as well as vague but related statements, topic shifts, refusals or declines to answer and answers directed at a different subquestion. It is instructed not to reward political rhetoric unless it answers the target question, not to infer facts absent from the answer and to accept an implicit answer only if it provides enough information to resolve the target question and is constrained to return exactly one JSON object with the fields described below and nothing else.

Output fields

- `relevant_answer_summary`: one sentence summarizing only the content relevant to the target question.
- `direct_answer_present`: boolean flag indicating whether a direct answer phrase is present.
- `direct_answer_text`: the exact short phrase if a direct answer is present, otherwise an empty string.
- `concrete_information`: list of concrete, specific relevant points (facts, actions, reasons, commitments).
- `vague_or_general_information`: list of relevant but vague or general points.
- `off_target_content`: list of content that answers a different subquestion or changes topic.
- `refusal_or_non_answer_signals`: list of signals indicating refusal or avoidance.

Why I chose this
A single classification pass tends to conflate "answered something, just vaguely" with "answered a completely different question," even though these point toward different labels (Ambivalent versus Clear Non-Reply) this motivates separating concrete_information from vague_or_general_information and explicitly isolating off_target_content as its own field. The instruction not to reward political rhetoric and not to infer unstated facts was chosen to keep this agent's extraction grounded strictly in what the answer actually says, preventing fluent but non responsive answers from being misread as substantive. Allowing implicit answers, but only when sufficient to resolve the target question, was meant to avoid penalizing indirect yet genuinely clear answers while still requiring real informational content. Finally, constraining the output to a single, strictly formatted JSON object was meant to give the Gap and Evasion Agent a concrete, parseable summary to compare against, instead of a paragraph it would have to re-interpret and to make downstream parsing more robust against malformed generations.

Gap and Evasion Agent

What I have
This agent receives the TARGET_QUESTION, the answer and the outputs of both the Question Intent Agent and the Answer Content Agent and compares what the target question requires with what the answer actually provides. It is guided to distinguish a minor gap (missing a small detail but mostly answering) from a partial gap (relevant information present but vague, incomplete, generic or indirect) and a major gap (mostly answering another topic, refusing or avoiding the target question). It is further guided to select the best fitting evasion category from a fixed taxonomy i.e. explicit_answer, implicit_answer, partial_half_answer, general_answer, dodging, deflection, declining_to_answer, claims_ignorance, clarification_only or irrelevant. Explicitly told not to assign the final top level label itself and is constrained to return exactly one JSON object with the fields described below and nothing else.

Output fields

- `coverage`: categorical level of how much of the target question's requirements are satisfied (complete, mostly_complete, partial, minimal, none).
- `gap_level`: categorical severity of the gap between what is required and what is provided (none, minor, partial, major).
- `best_evasion_category`: single best fitting category from the fixed evasion taxonomy.
- `missing_requirements`: list of specific requirements from the target question that the answer fails to address.
- `evidence_for_answering`: list of short evidence snippets from the answer supporting that it does answer the question.
- `evidence_for_evasion_or_gap`: list of short evidence snippets supporting that it evades or falls short of the question.
- `gap_explanation`: one concise sentence summarizing the gap assessment.

Why I chose this
This agent is the diagnostic core of the pipeline, so it was given both prior agents structured outputs rather than raw text, so that it compares an explicit requirement checklist against an explicit content extraction instead of rederiving both from the original question and answer. The three level gap distinction (minor/partial/major) was chosen because a binary answered/not answered judgment collapses exactly the cases the dataset cares about most, namely partially responsive or vague answers, into a single bucket, whereas a continuous severity scale gives the Decision Agent a finer signal to work with. The fixed ten category evasion taxonomy was chosen over open ended reasoning to make the agent's characterization of *how* an answer falls short consistent and reusable across examples, rather than producing a different ad-hoc description each time. Explicitly forbidding this agent from assigning the final label was meant to keep the diagnostic step and the decision step separate, so that label errors can be traced back to either a diagnostic error (wrong coverage/gap assessment) or a decision

rule error, rather than being entangled in a single opaque judgment. Finally, constraining the output to a single, strictly formatted JSON object was meant to give the Decision Agent a concrete, parseable diagnosis to apply its rules to, instead of a paragraph it would have to re-interpret, and to make downstream parsing more robust against malformed generations.

Decision Agent

What I have
This agent receives the TARGET_QUESTION, the answer and the outputs of all three preceding agents and assigns exactly one final label out of Clear Reply, Ambivalent or Clear Non-Reply, judging only the target question rather than the full interview question. It is given explicit definitions for each label: Clear Reply requires the answer to directly or implicitly resolve the target question (including implied yes/no answers, regardless of length or political framing). Ambivalent covers answers with some relevant but incomplete, vague, overly general or only partially responsive content. Clear Non-Reply covers answers that refuse, claim ignorance, redirect or otherwise provide little or no usable information for the target question. It is further given explicit decision rules mapping the Gap and Evasion Agent's coverage level to a label, together with two tie breaking preferences i.e.preferring Ambivalent over Clear Non-Reply when there is some relevant information and preferring Ambivalent over Clear Reply when specific stance, action or reasoning is missing. Also, is provided five worked few-shot calibration examples (Examples A to E) covering each label, including a case where the answer responds to the speaker's own attribute rather than the one asked about in the target question. It is constrained to return exactly one JSON object with the fields described below and nothing else.

Output fields

- `label`: the final predicted class (Clear Reply, Ambivalent, or Clear Non-Reply).
- `confidence`: a numeric confidence score for the assigned label.
- `main_reason`: one concise sentence explaining the basis for the decision.
- `used_rule`: a short description of which decision rule determined the label.

Why I chose this
Centralizing the final label decision in one agent, fed only the structured intermediate outputs of the prior three agents rather than raw free text, was meant to make the decision traceable back to a specific coverage or gap assessment rather than an opaque rereading of the original question and answer. Explicit label definitions were included because the three way distinction is inherently fuzzy at the boundaries, particularly between Ambivalent and the other two classes, so anchoring each label to concrete criteria was intended to reduce inconsistent judgments. The decision rules tying coverage levels to labels, together with the two tie breaking preferences, were added to give the agent a default resolution strategy for borderline cases rather than leaving it to infer one independently each time. The five few-shot examples were chosen to cover

one canonical case per label plus a deliberately tricky referent confusion case (Example E, where the answer addresses the speaker rather than the third party named in the question), since zero-shot label assignment on a small model is unreliable for this kind of fine grained distinction and worked examples were expected to anchor the model's judgment more effectively than rule statements alone. Finally, requesting a main_reason and used_rule alongside the label, rather than the label in isolation, was meant to support later error analysis by making each prediction selfdocumenting.

3.1.3. Pipeline Mechanics.

Sequencing and data flow
The four agents run strictly sequentially for each question–answer pair: Question Intent Agent → Answer Content Agent → Gap and Evasion Agent → Decision Agent. Rather than concatenating the raw outputs of previous agents as free text, each agent's structured JSON output is passed directly as input to the next agent in the chain, so that every downstream agent reasons over an explicit, parseable representation of the prior agents analysis rather than having to re-extract it from unstructured text.

Generation and decoding settings
All agents use greedy decoding (do_sample=False) with a fixed random seed, to ensure reproducible outputs across runs. The three intermediate agents (Question Intent, Answer Content, Gap and Evasion) are allotted max_new_tokens = 180, while the Decision Agent is allotted max_new_tokens = 90. This asymmetry reflects the differing nature of each agent's output i.e. the intermediate agents produce richer structured analysis, including multiple list fields and explanatory text, whereas the Decision Agent's output is deliberately short by design, consisting only of a label, a confidence score and two brief justification fields.

JSON parsing robustness
Each agent's raw generation is passed through a JSON extraction utility before being used downstream. The utility first strips any Markdown code fences (e.g. ```` ```json ```` blocks) that the model may emit around the output, then attempts to parse the cleaned text directly as JSON. If direct parsing fails, it falls back to a regex search for the first `{...}` block in the text and attempts to parse that. If this also fails, the utility falls back to returning a dictionary containing only the raw text under a `raw` key, rather than raising an exception. This layered fallback ensures that a single malformed or non-JSON generation from the model does not interrupt the pipeline, allowing the run to continue to the next agent or example.

Label normalization
The label field returned by the Decision Agent is passed through a normalization function before being used as the final prediction. This function maps numeric label codes, common aliases (e.g. "non-reply", "ambiguous", "partial"), and partial string matches to one of the three canonical labels (Clear Reply, Ambivalent, Clear Non-Reply). When the returned text does not match any recognized pattern, the function defaults to predicting *Ambivalent*. This is a substantive design decision rather than an incidental implementation detail since Ambivalent is also the majority class in the dataset, this

fallback could systematically bias unparseable or malformed Decision Agent outputs toward Ambivalent, a possibility we revisit in the Discussion when interpreting the observed per-class error patterns.

Reproducibility and fault tolerance
During inference, results are checkpointed to disk every 25 processed rows, so that long running evaluations (in particular full test-set inference) can survive interruptions without losing previously computed predictions. In addition, each example is processed inside a try/except block, if any agent in the pipeline raises an exception while processing a given question–answer pair, the system falls back to predicting *Ambivalent* for that example, together with an error message recorded as the rationale, rather than halting the entire evaluation run. This ensures that a single problematic input does not prevent the rest of the dataset from being processed.

3.1.4. Results.

Overall metrics

Split	Accuracy	Precision (weighted)	Recall (weighted)	F1 (weighted)	Macro F1
Validation ($n = 100$)	0.570	0.596	0.570	0.575	0.462
Test ($n = 308$)	0.562	0.560	0.562	0.558	0.387

Table 2: Overall D3-Agentic Prompting performance on the validation and test splits

Per-class metrics

Split	Class	Precision	Recall	F1	Support
Validation	Clear Reply	0.34	0.48	0.40	25
	Ambivalent	0.70	0.65	0.68	66
	Clear Non-Reply	0.50	0.22	0.31	9
Test	Clear Reply	0.33	0.39	0.36	79
	Ambivalent	0.69	0.68	0.68	206
	Clear Non-Reply	0.20	0.09	0.12	23

Table 3: Per-class precision, recall, F1, and support for D3-Agentic Prompting on the validation and test splits

3.1.5. Results Analysis and Explanation.

Overall accuracy (0.570 on validation, 0.562 on test) and weighted F1 (0.575 and 0.558, respectively) remain fairly stable between splits, suggesting the pipeline is not severely overfit to the 100 example validation subset. However, Macro F1 drops sharply from 0.462 to 0.387 between validation and test, a gap driven almost entirely by the Clear Non-Reply class, whose F1 falls from 0.31 (recall 0.22, $n = 9$) to 0.12 (recall 0.09,

$n = 23$). Given the very small validation support for this class, the validation-set Macro F1 is a noisy estimate the larger test set more reliably exposes how poorly the system handles this label.

The per-class pattern is consistent across both splits: Ambivalent is predicted comparatively well (F1 of 0.68 on both splits), while Clear Reply and Clear Non-Reply are both weak. Notably, Clear Reply shows higher recall than precision on both splits (0.48 vs. 0.34 on validation and 0.39 vs. 0.33 on test), indicating the system overpredicts this label, likely pulling genuine Ambivalent and Clear Non-Reply cases into it.

We attribute this pattern to two aspects of the pipeline design. First, the Decision Agent's rule set explicitly instructs it to *prefer Ambivalent over Clear Reply when specifics are missing*, but has no symmetric rule pushing borderline, lowcoverage cases toward Clear Non-Reply rather than Clear Reply, meaning that whenever the Gap and Evasion Agent's coverage field is even slightly overestimated (e.g. "partial" instead of "none"), the Decision Agent's rules route the example toward Clear Reply or Ambivalent rather than Clear Non-Reply. This directly explains Clear Non-Reply's low recall, the system is structurally biased toward assuming some coverage is present even when the answer is, in fact, evasive.

Second, the severe class imbalance in the underlying data interacts with the difficulty of the best_evasion_category field, which requires the Gap and Evasion Agent to select among ten categories rather than the three top level labels. On a 0.8B-parameter model, this fine grained categorical decision is the noisiest intermediate output in the pipeline and errors here propagate directly into the Decision Agent's final label, compounding the asymmetric decision rule described above and further suppressing Clear Non-Reply recall on the larger, more diverse test set.

It is worth noting that this baseline performance was achieved using the agent prompts and pipeline scaffolding adapted directly from the provided lab code, without any modifications of our own. The fact that this unmodified implementation already reaches weighted F1 scores of 0.575 on validation and 0.558 on test which we consider decent to fairly good results supports our earlier assessment that the lab's prompt design and pipeline structure are very well built and organised.

In the experiments that follow, we extend several of the agents described above with additional output fields and explicitly instruct each agent to make use of these new fields in its reasoning. The goal of these modifications is to test whether providing richer, more targeted intermediate information allows the pipeline to achieve better classification performance.

Note In Experiments 2 to 4, we build directly on the codebase from Experiment 1 and in each experiment we add new output fields to one agent and modify the prompt of the downstream agent that consumes that output, so that it explicitly makes use of the new fields.

3.2. Experiment 2 — Combination 1: Question Understanding

This experiment builds directly on the Experiment 1 codebase. Two agents were modified: the Question Intent Agent, whose output schema was extended with two

new fields and the Gap and Evasion Agent, whose prompt was revised to explicitly use those fields when judging coverage. The Answer Content Agent and the Decision Agent are unchanged from Experiment 1.

3.2.1. Question Intent Agent — Changes.

What I did
In addition to the fields already produced in Experiment 1, the Question Intent Agent's output schema was extended with two new fields describing properties of the target question itself: how specific the question is and whether it can be meaningfully answered with a yes or no. Specifically, the following two lines were added to the Question Intent Agent's JSON schema from Experiment 1:

```
"question_specificity": "very_specific|specific|broad|very_broad",
"yes_no_possible": true
```

Output fields

- `question_specificity`: categorical label for how specific the target question is (very_specific, specific, broad, very_broad).
- `yes_no_possible`: boolean flag indicating whether the target question can be meaningfully answered with yes or no.

Why I chose this
These two fields were chosen together because they both describe an intrinsic property of the question itself, rather than of the answer and work naturally as a combination because knowing how specific a question is and whether it admits a yes/no answer directly informs how strict the standard for "complete coverage" should be when an answer is later evaluated.

3.2.2. Gap and Evasion Agent — Changes.

What I did
The Gap and Evasion Agent's prompt was extended with an explicit calibration block instructing it to use question_specificity and yes_no_possible from the Question Intent Agent's output when judging coverage: if yes_no_possible is true and the answer clearly implies yes or no, this is treated as sufficient coverage even without elaboration. If question_specificity is *very_specific* or *specific*, a short, direct, on topic answer can still count as complete coverage and if question_specificity is *broad* or *very_broad*, a short answer is more likely to represent only partial coverage, since broad questions typically require more elaboration to be considered fully addressed. Specifically, the following block was added to the Gap and Evasion Agent's prompt from Experiment 1:

```
Use question_specificity and yes_no_possible from the Question Intent Agent
output to calibrate how strict your judgment should be:
- If yes_no_possible is true and the answer clearly implies yes or no,
  treat this as sufficient coverage even without elaboration.
- If question_specificity is very_specific or specific, a short, direct,
  on-topic answer can still count as complete coverage.
- If question_specificity is broad or very_broad, a short answer is more
  likely to be only partial coverage, since broad questions typically
  require more elaboration to be considered fully addressed.
```

Why I chose this
Generating the question_specificity and yes_no_possible fields in the Question Intent Agent's output is only useful if a downstream agent is explicitly instructed to act on them, otherwise, they would remain available as raw context without any defined role in the model's reasoning. For this reason, the Gap and Evasion Agent's prompt was revised to explicitly reference both fields and define concrete decision rules for their use, as described above, converting them from passively available context into actively applied reasoning signals that directly calibrate how strict the coverage judgment should be.

3.2.3. Generation Settings Change.

The token budgets for generation were increased relative to Experiment 1: max_new_tokens for the three intermediate agents was raised from 180 to 400 and for the Decision Agent from 90 to 180. This change was made to reduce the risk of truncated generations now that the Question Intent Agent's output includes additional fields.

3.2.4. Results.

Overall metrics

Split	Accuracy	Precision (weighted)	Recall (weighted)	F1 (weighted)	Macro F1
Validation ($n = 100$)	0.480	0.676	0.480	0.478	0.396
Test ($n = 308$)	0.435	0.550	0.435	0.441	0.342

Table 4: Overall D3-Agentic Prompting performance for Experiment 2 (Combination 1) on the validation and test splits

Per-class metrics

Split	Class	Precision	Recall	F1	Support
Validation	Clear Reply	0.32	0.84	0.47	25
	Ambivalent	0.76	0.39	0.52	66
	Clear Non-Reply	1.00	0.11	0.20	9
Test	Clear Reply	0.29	0.67	0.40	79
	Ambivalent	0.68	0.38	0.49	206
	Clear Non-Reply	0.29	0.09	0.13	23

Table 5: Per-class precision, recall, F1, and support for Experiment 2 (Combination 1) on the validation and test splits

3.2.5. Results Analysis and Explanation.

Compared to the Experiment 1 baseline, Experiment 2 performs substantially worse across every aggregate metric: accuracy falls from 0.570 to 0.480 on validation and from 0.562 to 0.435 on test, while Macro F1 falls from 0.462 to 0.396 on validation and from 0.387 to 0.342 on test. The calibration rules added to the Gap and Evasion Agent, intended to make coverage judgments more lenient for specific or yes/no questions, therefore had a net negative effect on overall classification quality.

The per-class breakdown shows precisely where this regression comes from. Clear Reply's recall rises sharply, from 0.48 to 0.84 on validation and from 0.39 to 0.67 on test, but its precision falls, from 0.34 to 0.32 on validation and from 0.33 to 0.29 on test. At the same time, Ambivalent's recall collapses, from 0.65 to 0.39 on validation and from 0.68 to 0.38 on test, even though its precision rises (0.70 to 0.76 on validation and 0.69 to 0.68 on test, essentially unchanged). This is a clear signature of a systematic shift in the decision boundary between Clear Reply and Ambivalent, the system is now predicting Clear Reply far more often and a large share of these new Clear Reply predictions are cases that were previously and more correctly, classified as Ambivalent.

We attribute this shift directly to the calibration rules added in this experiment. The instruction that a short, direct, on topic answer can count as complete coverage when the question is *specific* or *very_specific*, or when `yes_no_possible` is true, was intended to prevent the system from unfairly penalizing short but sufficient answers. In practice, however, this rule appears to have been applied too liberally by the Gap and Evasion Agent i.e. many answers that are short and on topic but still only partially responsive, the kind of case the dataset labels as Ambivalent, are now being judged as having "complete" coverage, which the Decision Agent's existing rules then map to Clear Reply. In other words, the new calibration logic lowered the bar for what counts as a sufficient answer more broadly than intended, rather than only for the narrow set of cases it was designed to target.

Clear Non-Reply shows a smaller but related effect, precision rises substantially (0.50 to 1.00 on validation and 0.20 to 0.29 on test), while recall remains very low on both splits. This suggests that the new rules did not directly cause additional Clear Non-Reply cases to be misclassified, since the rules concern when an answer should be considered *sufficient*, not when it should be considered evasive, but the class remains as hard to recover as in Experiment 1, since the underlying coverage-estimation problem

identified in the baseline Results Analysis is unaffected by this change.

Overall, Experiment 2 indicates that grounding the Gap and Evasion Agent's strictness in question-level properties (question_specificity, yes_no_possible) is a reasonable idea in principle, but the specific thresholds used here, treating any short, on topic answer to a specific or yes/no question as complete, are too permissive for a 0.8B model to apply with the intended nuance and instead degrade the model's ability to separate Clear Reply from Ambivalent, which was the best performing class in the baseline.

3.3. Experiment 3 — Combination 2: Answer Analysis

This experiment builds directly on the Experiment 1 codebase. Two agents were modified: the Answer Content Agent, whose output schema was extended with three new fields and the Gap and Evasion Agent, whose prompt was revised to use those fields as supporting evidence when judging coverage. The Question Intent Agent and the Decision Agent are unchanged from Experiment 1.

3.3.1. Answer Content Agent — Changes.

What I did
In addition to the fields already produced in Experiment 1, the Answer Content Agent's output schema was extended with three new fields describing properties of the answer itself: how relevant the answer is to the target question, whether a topic shift is detected and how complete the answer is. Specifically, the following three lines were added to the Answer Content Agent's JSON schema from Experiment 1:

```
"answer_relevance_score": "high|medium|low|none",
"topic_shift_detected": true,
"answer_completeness": "complete|partial|minimal|empty"
```

Output fields

- `answer_relevance_score`: categorical label for how relevant the answer is to the target question (high, medium, low, none).
- `topic_shift_detected`: boolean flag indicating whether the answer shifts away from the target question's topic.
- `answer_completeness`: categorical label for how complete the answer is on its own terms (complete, partial, minimal, empty).

Why I chose this
These three fields were chosen together because they all describe an intrinsic property of the answer itself, rather than of the question and work naturally as a combination, a relevance score, an explicit topic shift flag and a completeness judgment together give a more direct, multi angle signal of how substantively the answer engages with the target question than the Answer Content Agent's Experiment 1 fields alone.

3.3.2. Gap and Evasion Agent — Changes.

What I did
The Gap and Evasion Agent's prompt was extended with an explicit block instructing it to use answer_relevance_score, topic_shift_detected, and answer_completeness from the Answer Content Agent's output as supporting evidence, rather than as overriding rules, when judging coverage. Specifically, the following block was added to the Gap and Evasion Agent's prompt from Experiment 1:

```
Use answer_relevance_score, topic_shift_detected, and answer_completeness
from the Answer Content Agent output as additional evidence, but treat
them as supporting signals rather than overriding rules:
- If answer_relevance_score is low or none, this strongly suggests
  coverage should be partial or none, regardless of answer length.
- If topic_shift_detected is true, this is evidence toward dodging or
  deflection, even if the answer is long or fluent.
- If answer_completeness is minimal or empty, do not treat the answer
  as complete coverage even if it touches on the topic.
- These three signals should make your judgment more cautious about
  labeling an answer as complete, not less cautious. Do not use them
  to justify lowering the bar for what counts as a clear answer.
```

Why I chose this
Unlike the calibration rules added in Experiment 2, which explicitly lowered the bar for what counts as complete coverage in certain cases, this block was deliberately written to push the Gap and Evasion Agent in the opposite direction: the three new fields are framed only as evidence that should make the agent *more* cautious about labeling an answer as complete, never less cautious. This asymmetric framing was chosen specifically in response to the Experiment 2 finding that an overly permissive calibration rule degraded the system's ability to separate Clear Reply from Ambivalent, so this experiment tests whether a stricter, evidence based use of new fields avoids that failure mode while still benefiting from richer intermediate information.

3.3.3. Generation Settings.

The token budgets used in this experiment are unchanged from Experiment 2: max_new_tokens remains 400 for the three intermediate agents and 180 for the Decision Agent, since these budgets already accommodate the additional fields introduced here.

3.3.4. Results.

Overall metrics

Split	Accuracy	Precision (weighted)	Recall (weighted)	F1 (weighted)	Macro F1
Validation ($n = 100$)	0.400	0.503	0.400	0.399	0.331
Test ($n = 308$)	0.396	0.560	0.396	0.395	0.331

Table 6: Overall D3-Agentic Prompting performance for Experiment 3 (Combination 2) on the validation and test splits

Per-class metrics

Split	Class	Precision	Recall	F1	Support
Validation	Clear Reply	0.28	0.68	0.40	25
	Ambivalent	0.61	0.33	0.43	66
	Clear Non-Reply	0.33	0.11	0.17	9
Test	Clear Reply	0.28	0.73	0.40	79
	Ambivalent	0.70	0.30	0.42	206
	Clear Non-Reply	0.27	0.13	0.18	23

Table 7: Per-class precision, recall, F1, and support for Experiment 3 (Combination 2) on the validation and test splits

3.3.5. Results Analysis and Explanation.

Compared to both Experiment 1 and Experiment 2, Experiment 3 performs worse across every aggregate metric and is in fact the weakest of the three so far: accuracy falls to 0.400 on validation and 0.396 on test, weighted F1 falls to 0.399 and 0.395 and Macro F1 falls to 0.331 on both splits. This is a notably counter intuitive outcome, since unlike Experiment 2, the new instructions added here were explicitly framed to make the Gap and Evasion Agent *more* cautious about labeling an answer as complete, not less, yet the result is an even larger drop in performance.

The per-class breakdown shows a pattern that is, in direction, similar to Experiment 2 but more severe. Clear Reply's recall rises further still, to 0.68 on validation and 0.73 on test, while its precision remains essentially as low as in Experiment 2 (0.28 on both splits). Ambivalent's recall drops even lower than in Experiment 2, to 0.33 on validation and 0.30 on test, while Ambivalent's precision falls for the first time across experiments, from 0.76 in Experiment 2 to 0.61 on validation and from 0.68 to 0.70 on test, remaining roughly flat there. The net effect is that Clear Reply over prediction is now the dominant error mode of the whole pipeline, even more so than in Experiment 2.

One plausible explanation lies in the direct producer to consumer relationship between these two agents, rather than in the explicit caution instructions failing outright. The Answer Content Agent is the only point in the pipeline where the values of answer_relevance_score, topic_shift_detected and answer_completeness are actually decided and therefore the only point where they can be wrong. The Gap and Evasion Agent, immediately downstream, receives these values as fixed input i.e. what

I mean is it can only reason about what to do *given* them, with no mechanism to second guess or correct them if they are inaccurate. In other words, error can only enter the pipeline at the generation step and once generated, the Gap and Evasion Agent takes these values at face value rather than reverifying them, since the two agents are connected by a direct, single hop relationship with no intermediate agent capable of catching such errors. We were not able to confirm directly from row level agent outputs whether the Answer Content Agent in fact misjudged these fields too liberally, but this producer-consumer structure would explain how adding three additional categorical fields could introduce new points of error that propagate forward into the Gap and Evasion Agent's reasoning, regardless of how the consuming prompt is worded.

Clear Non-Reply shows mixed movement relative to Experiment 2: recall improves slightly on test (0.09 to 0.13) but precision drops on both splits (1.00 to 0.33 on validation; 0.29 to 0.27 on test, roughly unchanged). The validation precision drop is especially large, though the validation support for this class remains very small ($n = 9$), so this single split swing should be interpreted cautiously rather than as a robust trend.

Overall, Experiment 3 suggests that the failure mode identified in Experiment 2, which is the system overpredicting Clear Reply at Ambivalent's expense, is not specific to the particular calibration rule used there, since a deliberately stricter, more cautious instruction produces the same direction of error, only worse. This points to the added schema complexity itself, rather than the wording of the consuming agent's instructions, as a more likely source of the regression i.e. each new categorical field is itself a potential point of failure for a 0.8B model and simply telling a downstream agent to be cautious when using these fields does not protect against the fields themselves being generated inaccurately.

3.4. Experiment 4 (Best Experiment) — Combination 3: Pipeline Connection

This experiment builds directly on the Experiment 1 codebase. Two agents were modified: the Gap and Evasion Agent, whose output schema was extended with one new field and the Decision Agent, whose prompt was revised to use that field as a supporting signal and to explicitly summarize the key signals from previous agents before assigning the final label. The Question Intent Agent and the Answer Content Agent are unchanged from Experiment 1.

3.4.1. Gap and Evasion Agent — Changes.

What I did
In addition to the fields already produced in Experiment 1, the Gap and Evasion Agent's output schema was extended with one new field summarizing its overall verdict on whether the answer addresses the target question. Specifically, the following line was added to the Gap and Evasion Agent's JSON schema from Experiment 1:

```
"overall_verdict": "answers|partially_answers|does_not_answer"
```

Output fields

- `overall_verdict`: a single, top-level summary judgment of whether the answer addresses the target question (answers, partially_answers, does_not_answer).

Why I chose this
Unlike the fields added in Experiments 2 and 3, which described properties of the question or the answer separately, overall_verdict is designed to connect the pipeline together i.e. it condenses the Gap and Evasion Agent's own detailed coverage and gap_level judgments into a single, label aligned verdict that the Decision Agent can use directly, rather than having to rederive a toplevel judgment from the more granular fields itself every time.

3.4.2. Decision Agent — Changes.

What I did
The Decision Agent's prompt was extended with an explicit block instructing it to use overall_verdict from the Gap and Evasion Agent's output as a direct, supporting signal for the final label, while still prioritizing the more detailed coverage and gap_level fields if the two disagree. The Decision Agent was further instructed to first summarize the key signals from the previous agents before assigning the final label and its output schema was extended with a corresponding key_signals_used field. Specifically, the following blocks were added to the Decision Agent's prompt and schema from Experiment 1:

```
Use overall_verdict from the Gap and Evasion Agent output as a direct
signal for your label, but do not let it override the detailed
coverage and gap_level fields:
- If overall_verdict is "answers", this supports Clear Reply,
  consistent with the coverage and gap_level fields.
- If overall_verdict is "partially_answers", this supports Ambivalent.
- If overall_verdict is "does_not_answer", this supports Clear
  Non-Reply.
- Use overall_verdict to double-check consistency with coverage and
  gap_level; if they disagree, prioritize the more detailed coverage
  and gap_level fields over the single-word overall_verdict.

Before assigning the final label, first summarize the key signals
from the previous agents.
```

```
"key_signals_used": {
  "coverage": "complete|mostly_complete|partial|minimal|none",
  "gap_level": "none|minor|partial|major",
  "overall_verdict": "answers|partially_answers|does_not_answer"
}
```

Why I chose this
This change was designed to make the Decision Agent's reasoning more explicitly grounded in the intermediate pipeline outputs, rather than implicitly absorbing them. Requiring the agent to restate coverage, gap_level and overall_verdict as a key_signals_used object before producing the label is intended to act as a lightweight, selfimposed check, forcing the model to explicitly commit to which signals it is relying on rather than jumping directly to a label. Treating overall_verdict as a secondary, consistency checking signal rather than a primary one was a deliberate choice to avoid the failure mode observed in Experiment 2, where giving a new field too much direct influence over the label led to systematic overprediction of a single class.

3.4.3. Generation Settings.

The token budgets used in this experiment are unchanged from Experiments 2 and 3: max_new_tokens remains 400 for the three intermediate agents and 180 for the Decision Agent.

3.4.4. Results.

Overall metrics

Split	Accuracy	Precision (weighted)	Recall (weighted)	F1 (weighted)	Macro F1
Validation ($n = 100$)	0.440	0.512	0.440	0.458	0.339
Test ($n = 308$)	0.578	0.638	0.578	0.592	0.478

Table 8: Overall D3-Agentic Prompting performance for Experiment 4 (Combination 3) on the validation and test splits

Per-class metrics

Split	Class	Precision	Recall	F1	Support
Validation	Clear Reply	0.23	0.44	0.31	25
	Ambivalent	0.65	0.48	0.56	66
	Clear Non-Reply	0.25	0.11	0.15	9
Test	Clear Reply	0.37	0.62	0.47	79
	Ambivalent	0.77	0.60	0.67	206
	Clear Non-Reply	0.33	0.26	0.29	23

Table 9: Per-class precision, recall, F1, and support for Experiment 4 (Combination 3) on the validation and test splits

3.4.5. Results Analysis and Explanation.

Experiment 4 produces the strongest test-set result of all four experiments and the only one to clearly exceed the Experiment 1 baseline. Test accuracy rises from 0.562 (Experiment 1) to 0.578, weighted F1 rises from 0.558 to 0.592 and most notably, Macro F1 rises from 0.387 to 0.478, a substantial improvement of more than nine points. This reverses the regressions seen in Experiments 2 and 3, where every aggregate metric fell relative to the baseline.

Validation results, however, tell a different story: accuracy (0.440) and Macro F1 (0.339) remain below the Experiment 1 baseline (0.570 and 0.462, respectively), continuing the pattern seen in Experiments 2 and 3. This validation-test divergence is the largest of any experiment so far and is itself worth treating as a finding, with only $n = 100$ validation examples and an even smaller $n = 9$ for Clear Non-Reply specifically, the validation split appears to be a noisy and potentially unrepresentative estimate of test-set behavior for this pipeline, more so than the relatively small gaps observed in Experiments 1 through 3.

The per-class test results show broad improvement across all three classes simultaneously, which did not occur in any previous experiment. Clear Non-Reply's F1 more than doubles relative to the baseline, from 0.12 to 0.29, with recall rising from 0.09 to 0.26, the best Clear Non-Reply recall observed in any experiment. Ambivalent's F1 also improves slightly, from 0.68 to 0.67 is essentially unchanged, while precision rises from 0.69 to 0.77. Clear Reply's F1 improves from 0.36 to 0.47, with recall rising from 0.39 to 0.62, though its precision remains the weakest of the three classes at 0.37.

We attribute this improvement to two related design choices: treating overall_verdict as a secondary, consistency-checking signal rather than a primary one and explicitly requiring the Decision Agent to restate coverage, gap_level and overall_verdict as a key_signals_used object before producing its label. A key difference from Experiments 2 and 3 is that overall_verdict is not an independent new property of the question or answer, but a condensed summary of judgments the Gap and Evasion Agent already makes (coverage, gap_level), the Decision Agent is therefore reasoning over a restatement of existing signals rather than an entirely new categorical distinction it must learn to interpret from scratch. This is consistent with a tentative pattern across Experiments 2 through 4, fields that summarize or make an agent's own existing reasoning more explicit (Experiment 4) appear to be safer additions than fields that introduce a genuinely new, independently judged property for a downstream agent to interpret (Experiments 2 and 3), since the latter adds a fresh opportunity for a small 0.8B model to misjudge an unfamiliar categorical distinction. We note this as a pattern observed across only three experiments and not as a confirmed general principle.

Clear Non-Reply's improved recall is particularly notable given that this was the weakest class identified in the Experiment 1 Results Analysis. We were not able to confirm this directly from row-level agent outputs, but a plausible mechanism is that requiring the Decision Agent to explicitly restate gap_level and overall_verdict together in key_signals_used makes it harder for the model to default to Clear Reply or Ambivalent when these two fields both indicate a major gap or a does_not_answer verdict, since the model must now explicitly write out signals that point toward Clear Non-Reply rather than implicitly overlooking them.

4. Comparison with Previous Assignments

For this comparison, the best performing experiment from each previous assignment is used, rather than the baseline, so we will compare the best experiments for each assignment.

4.1. Vector-Based Classifier (Assignment 1)

Overview
In the first assignment, the best performing model was a Word2Vec representation combined with Logistic Regression. Because I had much more time to give for this assignment, I was trying to find a way to get better scores, which led me, through searching online, to this link here: https://www.kaggle.com/code/kstathou/word-embeddings-logistic-regression, suggesting that Word2Vec should be trained from scratch on the project's own dataset rather than using a pre-trained vocabulary such as Google News Word2Vec. Adopting this approach produced a notable improvement in performance, since the model could learn word representations specifically tailored to the vocabulary and context of political interview language, rather than relying on the more general vocabulary that came from Google News.

Note on Evaluation Metric
At the time Assignment 1 was completed, I did not know that the Kaggle competition's primary ranking metric was weighted F1 rather than macro F1, so the original report for that assignment documented only macro F1 as the evaluation metric. Since weighted F1 is computed from the same test-set predictions, this value is identical whether obtained locally or read from the Kaggle leaderboard, for this reason, the weighted F1 score observed on the Kaggle leaderboard for this submission (0.58) is reported alongside the originally documented macro F1 score in the comparison below. Weighted precision and weighted recall were not computed at the time and are therefore not reported for this model.

Overall metrics

Split	Accuracy	F1 (weighted)	Macro F1
Validation ($n = 507$)	0.59	–	0.4689
Test ($n = 308$)	0.64	0.58	0.4775

Table 10: Validation and test performance of the best Assignment 1 Experiment (Word2Vec + Logistic Regression)

Per-class metrics

Split	Class	Precision	Recall	F1	Support
Validation	Clear Reply	0.51	0.31	0.39	154
	Ambivalent	0.65	0.77	0.71	301
	Clear Non-Reply	0.30	0.33	0.31	52
Test	Clear Reply	0.36	0.20	0.26	79
	Ambivalent	0.71	0.84	0.77	206
	Clear Non-Reply	0.41	0.39	0.40	23

Table 11: Per-class precision, recall, F1 and support for the best Assignment 1 Experiment (Word2Vec + Logistic Regression) on the validation and test splits

4.2. Encoder Only Transformer Model (Assignment 2)

Overview
In the second assignment, three encoder only transformer models were finetuned for the same three class clarity classification task: BERT (bert-base-uncased), DistilBERT (distilbert-base-uncased), and DeBERTa-v3-base. The best experiment was BERT (Experiment 4), which achieved the best overall result and is used as the representative encoder-only model in the comparison that follows.

A key challenge with this dataset is that many combined question-answer sequences exceed the 512 token limit supported by these models, with approximately 30% of the dataset exceeding this limit when the question and answer are concatenated. If the question and answer were simply concatenated a large amount of information from the answer would be lost, since it would be cut off for exceeding the 512 token limit, leaving the models unable to make a properly informed prediction. To address this, a custom truncation strategy was designed, specifically if the question exceeded 130 tokens it was chopped off, otherwise it was kept in full and if the question did not take up the full 130 tokens, the remaining tokens went to the answer, so the answer would have 512 - 130 tokens plus whatever was left over from the question's 130 token allowance. The answer itself was then split using a head + tail strategy, keeping tokens from both the beginning and the end of the answer rather than only one side. This was found to consistently outperform head-only or tail-only truncation across all three models. This suggests that the conclusion of a political answer also carries important clarity signals, for example a politician may start ambiguously but give a clearer or more evasive conclusion at the end. By keeping only the beginning of the answer the model loses access to this information.

Note on Validation Metrics
For the best BERT configuration (Experiment 4), only the validation F1 (weighted) was recorded (0.6694), the validation accuracy, validation macro F1 and a validation per-class breakdown were not separately reported and are therefore not included below. Full metrics are available for the test set.

Overall metrics

Split	Accuracy	F1 (weighted)	Macro F1
Validation	–	0.6694	–
Test	0.6461	0.6500	0.5809

Table 12: Validation and test performance of the best Assignment 2 experiment (BERT, Experiment 4)

Per-class metrics (Test set)

Class	Precision	Recall	F1
Clear Reply	0.41	0.44	0.43
Ambivalent	0.76	0.73	0.74
Clear Non-Reply	0.54	0.61	0.57

Table 13: Per-class precision, recall, and F1 for the best Assignment 2 experiment (BERT, Experiment 4) on the test split

4.3. Single LLM Prompting (Assignment 3)

Overview
Unlike the first two assignments, I did not focus as much time on this assignment as I did on Assignments 1 and 2, where I specifically pushed to extract the best possible score. The result obtained here, a test weighted F1 of 0.4844, was considered a decent score for this assignment.

In this assignment, three instruction-tuned models from the Qwen3.5 family (Qwen3.5-0.8B, Qwen3.5-2B, Qwen3.5-4B) were evaluated using a single-prompt classification approach, without any agent decomposition or fine-tuning. Six prompting strategies were tested across all three models: zero-shot prompting, few-shot prompting, chain-of-thought (CoT) prompting, persona prompting, and two hybrid strategies combining persona with few-shot and persona with CoT. Each strategy was evaluated both with and without the sampling parameters recommended on each model's official Hugging Face model card and across six sample sizes ranging from 100 to 500 examples. This was because of the small sample size of 100, which meant the results of the initial experiment may not have been fully reliable or representative of true model performance. For this reason, subsequent experiments were conducted with larger sample sizes to obtain more statistically reliable results and to be more confident that the observed differences between models and strategies were genuine.

Best Strategy
The best performing combination overall was Qwen3.5-0.8B with few-shot prompting

(Experiment 11), using a labeled example per class extended with an explicit Reason field explaining why each label was chosen, combined with the recommended model card sampling parameters and 400 training samples. This configuration achieved a validation weighted F1 of 0.4274 and a test weighted F1 of 0.4844, the highest test score across all 13 experiments conducted. The addition of the Reason field to each few-shot example was the single most impactful prompt modification, since it gave the model explicit reasoning behind each labeled example rather than just the example itself, which is consistent with the broader finding that the smallest model benefited most from explicit reasoning guidance, while larger models performed comparatively better with simpler prompting strategies.

Overall metrics

Split	F1 (weighted)	Macro F1
Validation	0.4274	0.2429
Test	0.4844	–

Table 14: Validation and test performance of the best Assignment 3 experiment (Qwen3.5-0.8B + few-shot, Experiment 11)

Per-class metrics

Split	Class	Precision	Recall	F1
Validation	Clear Reply	0.32	0.27	0.29
	Ambivalent	0.59	0.51	0.55
	Clear Non-Reply	0.10	0.20	0.13
Test	Clear Reply	0.34	0.32	0.33
	Ambivalent	0.67	0.51	0.58
	Clear Non-Reply	0.11	0.35	0.17

Table 15: Per-class precision, recall, and F1 for the best Assignment 3 experiment (Qwen3.5-0.8B + few-shot, Experiment 11) on the validation and test splits

5. Overall Comparison Across Assignments

5.1. Combined Summary

Table 16 presents the test set performance of the best experiment from each of the four assignments, while Table 17 presents the corresponding validation set performance.

Approach	Accuracy	F1 (weighted)	Macro F1
Vector-Based Classifier (Word2Vec + LR)	0.64	0.58	0.4775
Encoder-Only Transformer (BERT)	0.6461	0.6500	0.5809
Single LLM Prompting (Qwen3.5-0.8B + few-shot)	–	0.4844	–
D3-Agentic Prompting (Experiment 4)	0.578	0.592	0.478

Table 16: Test set performance comparison of the best experiment from each approach

Approach	Accuracy	F1 (weighted)	Macro F1
Vector-Based Classifier (Word2Vec + LR)	0.59	–	0.4689
Encoder-Only Transformer (BERT)	–	0.6694	–
Single LLM Prompting (Qwen3.5-0.8B + few-shot)	–	0.4274	0.2429
D3-Agentic Prompting (Experiment 4)	0.440	0.458	0.339

Table 17: Validation set performance comparison of the best experiment from each approach

5.2. Per-Class Comparison (Test Set)

Table 18 presents the per-class precision, recall, and F1 on the test set for the best experiment from each of the four assignments.

Approach	Class	Precision	Recall	F1
Vector-Based Classifier (Word2Vec + LR)	Clear Reply	0.36	0.20	0.26
	Ambivalent	0.71	0.84	0.77
	Clear Non-Reply	0.41	0.39	0.40
Encoder-Only Transformer (BERT)	Clear Reply	0.41	0.44	0.43
	Ambivalent	0.76	0.73	0.74
	Clear Non-Reply	0.54	0.61	0.57
Single LLM Prompting (Qwen3.5-0.8B + few-shot)	Clear Reply	0.34	0.32	0.33
	Ambivalent	0.67	0.51	0.58
	Clear Non-Reply	0.11	0.35	0.17
D3-Agentic Prompting (Experiment 4)	Clear Reply	0.37	0.62	0.47
	Ambivalent	0.77	0.60	0.67
	Clear Non-Reply	0.33	0.26	0.29

Table 18: Per-class precision, recall, and F1 on the test set for the best experiment from each approach

5.3. Failure Part Comparison

A central question for this comparison is whether the four approaches share the same weaknesses or fail in different, complementary ways. Since each approach was developed and evaluated in a separate assignment, the comparison below is based on the aggregate per-class patterns shown in Table 18.

Ambivalent is the one class on which all four approaches agree, it is the easiest class for every approach, with F1 ranging from 0.58 (Single LLM Prompting) to 0.77 (D3-Agentic Prompting). This is consistent across all four assignments and is unsurprising given that Ambivalent is the dominant class in the training data for every approach, giving each model substantially more exposure to this class than to either minority class.

Beyond Ambivalent, however, the four approaches do *not* share a single common weak spot. Clear Non-Reply ranges from a near collapse at F1=0.17 (Single LLM Prompting) to a comparatively strong F1=0.57 (Encoder-Only Transformer), while Clear Reply ranges from F1=0.26 (Vector Based Classifier) to F1=0.47 (D3-Agentic Prompting). Notably, the relative ordering of the two minority classes flips between approaches i.e. for the Vector Based Classifier, Single LLM Prompting, and Encoder-Only Transformer, Clear Non-Reply is at least as strong as, or stronger than, Clear Reply, but for D3-Agentic Prompting, this relationship reverses, with Clear Reply (F1=0.47) clearly outperforming Clear Non-Reply (F1=0.29). This reversal makes sense because of the bias identified in the D3-Agentic Results Analysis (Experiment 4), the Decision Agent's rules have no symmetric mechanism pushing borderline, low coverage cases toward Clear Non-Reply, so whenever the Gap and Evasion Agent's coverage estimate is even slightly too generous, the pipeline routes the example toward Clear Reply or Ambivalent instead, inflating Clear Reply's recovery at Clear Non-Reply's expense, a structural bias that the other three approaches, lacking this explicit agentic decision logic, do not share. This therefore provides a concrete explanation for why this specific reversal occurs only in D3-Agentic Prompting and not in the other three approaches.

Looking at each approach individually, the nature of the minority class weakness differs in each case. The Vector-Based Classifier fails most severely on Clear Reply (F1=0.26), while still recovering a moderate share of Clear Non-Reply cases (F1=0.40). The Encoder-Only Transformer is the most balanced of the four, with both minority classes well above the others (F1=0.43 and F1=0.57), suggesting fine-tuning on the full dataset gives it more reliable exposure to both minority patterns. Single LLM Prompting shows the most extreme imbalance of any approach, with Clear Non-Reply nearly collapsing (F1=0.17) despite a comparatively higher recall driven by overpredicting this class, while Clear Reply remains only moderately better (F1=0.33). D3-Agentic Prompting is the only approach where Clear Reply is clearly the stronger of the two minority classes (F1=0.47 vs. F1=0.29), the reverse of the pattern seen in the other three approaches.

This comparison shows that while all four approaches share the same easy class (Ambivalent), each fails on the minority classes in a distinct way rather than converging on a single shared weakness.

5.4. Best Performing Approach

Based on the test-set results in Table 16, the Encoder-Only Transformer (BERT, Experiment 4) is the best performing approach overall, achieving the highest test weighted F1 (0.6500) and the highest test Macro F1 (0.5809) of all four approaches developed across this project, ahead of D3-Agentic Prompting (0.592 / 0.478), the Vector-Based Classifier (0.58 / 0.4775) and Single LLM Prompting (0.4844).

BERT's advantage is the result of several different factors, not a single cause. One

factor is the difference between fine-tuning and prompting, i.e. BERT's weights are directly updated during training on this task's labeled data, so the model learns decision boundaries between the three clarity labels as part of training itself. The two prompting-based approaches, Single LLM Prompting and D3-Agentic Prompting, never update the underlying Qwen3.5-0.8B model's weights at all, the model only ever encounters the task through the text of the prompt at inference time and must infer the classification logic from that prompt alone, without ever being trained specifically for this task. This is a structurally different and, on its own, harder problem to solve well. However, as discussed below, this is only part of the explanation, the difference in time invested in each assignment is at least as important.

It is also worth being transparent about the relative time investment behind these two results. Significantly more time was spent on the Encoder-Only Transformer assignment than on this D3-Agentic Prompting Assignment, including many more experiments that led to the design and tuning of the custom head + tail truncation strategy described earlier, which was a significant improvement over naive truncation. By contrast, the D3-Agentic Prompting baseline (Experiment 1) was adapted directly from the provided lab code with no modifications and a single, relatively small change in Experiment 4 (the addition of overall_verdict and the explicit key_signals_used restatement) was enough to raise the test weighted F1 from 0.558 to 0.592. This suggests that the lab provided D3-Agentic scaffold is already well designed and with spending as much time on this Assignment as was spent on Assignment 2, the D3-Agentic Prompting results would likely improve substantially and could plausibly come close to, or even surpass, the results achieved in Assignment 2.

It is also worth noting that BERT achieves this result with substantially fewer parameters (110M) than either Qwen3.5-based approach (0.8B), underscoring that fine-tuning a smaller model directly on the target task outperformed prompting a larger model without any task specific weight updates, at least at this model scale.

D3-Agentic Prompting (test weighted F1 0.592) outperforms both Single LLM Prompting (0.4844) and the Vector-Based Classifier (0.58) on the primary metric. On Macro F1, it is essentially on par with the Vector-Based Classifier (0.478 vs. 0.4775) and ahead of Single LLM Prompting, for which no Macro F1 was reported. This is a notable result, unlike D3-Agentic Prompting, the Vector-Based Classifier does involve training its own parameters from scratch on this dataset's data, yet D3-Agentic Prompting still matches or slightly exceeds it on both metrics without any training or fine-tuning at all, relying purely on the structure of its prompts and pipeline. This suggests that the agentic decomposition itself carries a meaningful share of the information that the Vector-Based Classifier had to learn through training, indicating that task decomposition is a powerful alternative to parameter learning, not merely a weaker substitute for it. This also indicates that decomposing the classification task into specialized agents, while using the same small 0.8B model and the same lack of fine-tuning as Single LLM Prompting, recovers a meaningful share of the performance gap toward BERT, supporting the central hypothesis of this Assignment, that is, that task decomposition can improve a prompted small model's performance, even if it does not fully close the gap with a fine-tuned encoder model.